微前端详解

一、什么是微前端

1. 定义

微前端（Micro Frontends）是一种**将前端应用拆分成多个独立、可单独开发、测试、部署的小型应用**，然后在运行时组合成一个完整应用的架构模式。

类比后端的微服务架构：

- 微服务：把后端拆成多个独立服务
- 微前端：把前端拆成多个独立应用

2. 核心思想

```
┌──────────────────────────────────────────────────────────────────┐
|                          主应用（基座）                          |
|  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐           |
|  |   子应用 A    |  |   子应用 B    |  |   子应用 C    |           |
|  |   (Vue 2)    |  |   (React)    |  |   (Vue 3)    |           |
|  |   团队 A     |  |   团队 B     |  |   团队 C     |          |
|  └──────────────┘  └──────────────┘  └──────────────┘           |
└──────────────────────────────────────────────────────────────────┘

```

3. 核心价值

价值	说明
技术栈无关	各子应用可使用不同框架（Vue/React/Angular）
独立开发部署	各团队独立开发、测试、部署，互不影响
增量升级	渐进式重构老项目，新功能用新技术
独立运行时	各子应用状态隔离，不互相影响

二、为什么需要微前端

1. 解决的问题

巨石应用困境

```
传统巨石应用的问题：
├── 代码量庞大，构建慢（10分钟+）
├── 技术栈老旧，升级困难（牵一发动全身）
├── 多团队协作冲突（代码合并地狱）
├── 测试困难（改一处要全量回归）
└── 难以局部发布（必须整体部署）

```

微前端的解决方案

```
微前端拆分后：
├── 各子应用独立构建（1-2分钟）
├── 技术栈自由选择（新模块用新技术）
├── 团队独立开发（各管各的仓库）
├── 独立测试部署（只发布修改的子应用）
└── 故障隔离（一个子应用挂了不影响其他）

```

2. 适用场景

场景	说明
大型企业应用	多团队、多业务线协作
老项目渐进式重构	新功能用新技术，老功能逐步迁移
多产品整合	多个独立产品整合到统一门户
中台建设	通用能力抽取，多业务复用
跨团队协作	不同团队负责不同模块

3. 不适用场景

- 小型项目（杀鸡用牛刀）

- 对性能要求极高的应用（有额外开销）
- 团队技术栈统一且稳定
- 没有独立部署需求

三、微前端实现方案

1. 方案概览

方案	原理	代表	复杂度
iframe	天然隔离	原生	低
NPM 包	构建时集成	组件库	低
路由分发	Nginx 转发	Nginx	低
Web Components	自定义元素	原生	中
JS 沙箱 + 动态加载	运行时隔离	qiankun/micro-app	高
Module Federation	Webpack 5 模块联邦	webpack	中

---

2. 方案一：iframe

原理

- 使用 `<iframe>` 嵌入子应用
- 天然具备最强的隔离性

实现

```
<!-- 主应用 -->
<div id="app">
  <nav>主应用导航</nav>
  <div id="micro-container">
    <iframe
      src="http://child.example.com"
      frameborder="0"
      id="child-app"
    ></iframe>
  </div>
```

```
</div>

```

```
// 动态切换子应用
function loadApp(url) {
  const iframe = document.getElementById('child-app')
  iframe.src = url
}

// 父子通信
// 父 -> 子
document.getElementById('child-app').contentWindow.postMessage(
  { type: 'FROM_PARENT', data: 'hello' },
  'http://child.example.com'
)

// 子 -> 父
window.parent.postMessage(
  { type: 'FROM_CHILD', data: 'world' },
  'http://parent.example.com'
)

// 监听消息
window.addEventListener('message', (event) => {
  if (event.origin !== 'http://trusted.example.com') return
  console.log(event.data)
})

```

优点

- 实现简单
- 天然隔离（JS、CSS、DOM 完全隔离）
- 安全性高

缺点

- **用户体验差**：
  - 页面刷新 iframe 状态丢失
  - URL 不同步（浏览器前进后退问题）

    - 弹窗只能在 iframe 内部显示
    - 滚动条独立，双滚动条问题
- **性能问题**：每个 iframe 独立加载资源
- **通信复杂**：只能通过 `postMessage`
- SEO 不友好

适用场景

- 对隔离性要求极高
- 临时方案 / 快速集成第三方系统
- 不追求极致用户体验

---

3. 方案二：NPM 包集成

原理

- 子应用打包成 NPM 包
- 主应用安装依赖，构建时集成

实现

```
# 子应用发布
npm publish @company/sub-app-a

# 主应用安装
npm install @company/sub-app-a

```

```
// 主应用使用
import SubAppA from '@company/sub-app-a'

// Vue 中注册
app.use(SubAppA)

```

优点

- 实现简单

- 无运行时开销
- 类型提示友好

缺点

- **不是真正的"独立部署"**：子应用更新需要主应用重新构建
- 版本管理复杂
- 构建时间变长

适用场景

- 基础组件库 / 业务组件库
- 更新频率低的模块
- 对独立部署要求不高

---

4. 方案三：路由分发（MPA）

原理

- 通过 Nginx 根据 URL 路由到不同应用
- 本质是多个独立应用，通过 URL 串联

Nginx 配置

```
server {
    listen 80;
    server_name example.com;

    # 主应用
    location / {
        proxy_pass http://main-app:3000;
    }

    # 子应用 A
    location /app-a/ {
        proxy_pass http://sub-app-a:3001/;
    }

    # 子应用 B
    location /app-b/ {
        proxy_pass http://sub-app-b:3002/;
```

```
    }
}

```

优点

- 实现最简单
- 完全独立部署
- 技术栈完全无关

缺点

- 切换应用时**整页刷新**
- 无法共享状态（除非用 localStorage / Cookie）
- 公共资源无法复用

适用场景

- 对用户体验要求不高
- 各应用关联性弱
- 快速落地

---

5. 方案四：Web Components

原理

- 使用 Custom Elements + Shadow DOM
- 子应用封装成自定义元素

实现

```
// 子应用：定义 Web Component
class MicroAppA extends HTMLElement {
  constructor() {
    super()
    // 创建 Shadow DOM（样式隔离）
    this.attachShadow({ mode: 'open' })
  }

  connectedCallback() {
    // 挂载时执行
```

```
    this.shadowRoot.innerHTML = `
      <style>
        .container { padding: 20px; }
      </style>
      <div class="container">
        <h1>子应用 A</h1>
      </div>
    `
    this.initApp()
  }

  disconnectedCallback() {
    // 卸载时执行
    this.destroyApp()
  }

  initApp() {
    // 初始化 Vue/React 应用到 shadowRoot
  }

  destroyApp() {
    // 销毁应用
  }
}

customElements.define('micro-app-a', MicroAppA)

```

```
<!-- 主应用使用 -->
<micro-app-a></micro-app-a>

```

优点

- 浏览器原生支持
- Shadow DOM 提供样式隔离
- 标准化方案

缺点

- 兼容性问题（需要 polyfill）
- Shadow DOM 内部框架支持有问题
- 调试困难

---

6. 方案五：JS 沙箱 + 动态加载（主流方案）

原理

```
1. 主应用加载子应用的入口文件（HTML/JS）
2. 解析并执行子应用代码
3. 通过 JS 沙箱隔离子应用的全局变量
4. 通过 CSS 隔离方案隔离样式
5. 管理子应用生命周期（挂载、卸载）


```

代表框架

- **qiankun**（蚂蚁）
- **micro-app**（京东）
- **wujie**（腾讯）
- **single-spa**（社区）
- **garfish**（字节）

---

7. 方案六：Module Federation（模块联邦）

原理

- Webpack 5 内置功能
- 运行时动态加载其他应用的模块
- 共享依赖，避免重复加载

配置示例

```
// 子应用 webpack.config.js（暴露模块）
const { ModuleFederationPlugin } = require('webpack').container

```

```
module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: 'subApp',
      filename: 'remoteEntry.js',
      exposes: {
        './Button': './src/components/Button',
        './App': './src/App',
      },
      shared: {
        react: { singleton: true },
        'react-dom': { singleton: true },
      },
    }),
  ],
}

```

```
// 主应用 webpack.config.js（消费模块）
module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: 'mainApp',
      remotes: {
        subApp: 'subApp@http://localhost:3001/remoteEntry.js',
      },
      shared: {
        react: { singleton: true },
        'react-dom': { singleton: true },
      },
    }),
  ],
}

```

```
// 主应用使用
import React, { lazy, Suspense } from 'react'

```

```
const RemoteButton = lazy(() => import('subApp/Button'))

function App() {
  return (
    <Suspense fallback="loading...">
      <RemoteButton />
    </Suspense>
  )
}

```

Vite 中使用（vite-plugin-federation）

```
// vite.config.ts
import federation from '@originjs/vite-plugin-federation'

export default defineConfig({
  plugins: [
    vue(),
    federation({
      name: 'sub-app',
      filename: 'remoteEntry.js',
      exposes: {
        './Button': './src/components/Button.vue',
      },
      shared: ['vue'],
    }),
  ],
})

```

优点

- Webpack 官方支持
- 依赖共享，减少重复加载
- 粒度细（可以共享组件级别）

缺点

- 强依赖 Webpack 5+
- 版本管理复杂
- 没有沙箱隔离

四、主流微前端框架详解

1. qiankun（蚂蚁）

基本信息

- 基于 single-spa 封装
- 最成熟的微前端方案之一
- HTML Entry 方式加载子应用

主应用配置

```
// main.ts
import { registerMicroApps, start, setDefaultMountApp } from 'qiankun'

// 注册子应用
registerMicroApps([
  {
    name: 'vue-app',           // 子应用名称
    entry: '//localhost:8081', // 子应用入口（HTML Entry）
    container: '#subapp',      // 挂载容器
    activeRule: '/vue',        // 激活路由
    props: {                   // 传递给子应用的数据
      token: 'xxx',
    },
  },
  {
    name: 'react-app',
    entry: '//localhost:8082',
    container: '#subapp',
    activeRule: '/react',
  },
])

// 设置默认子应用
setDefaultMountApp('/vue')
```

```

// 启动
start({
  prefetch: 'all',        // 预加载策略
  sandbox: {
    strictStyleIsolation: true,  // 严格样式隔离（Shadow DOM）
    // experimentalStyleIsolation: true, // 实验性样式隔离（Scoped CSS）
  },
})

```

```
<!-- 主应用 HTML -->
<div id="app">
  <nav>主应用导航</nav>
  <div id="subapp"></div>
</div>

```

子应用配置（Vue 3）

```
// main.ts
import { createApp, type App as VueApp } from 'vue'
import { createRouter, createWebHistory } from 'vue-router'
import App from './App.vue'
import routes from './router'

let app: VueApp<Element> | null = null
let router: ReturnType<typeof createRouter> | null = null

// 渲染函数
function render(props: any = {}) {
  const { container } = props

  router = createRouter({
    // 注意：子应用的 base 要加上 activeRule
    history: createWebHistory(window.__POWERED_BY_QIANKUN__ ? '/vue/' : '/'),
    routes,
```

```
  })

  app = createApp(App)
  app.use(router)

  // 挂载到 qiankun 提供的容器或默认 #app
  app.mount(container ? container.querySelector('#app') : '#app')
}

// 独立运行时
if (!window.__POWERED_BY_QIANKUN__) {
  render()
}

// 生命周期钩子 - 必须导出
export async function bootstrap() {
  console.log('[vue] bootstraped')
}

export async function mount(props: any) {
  console.log('[vue] mount', props)
  render(props)
}

export async function unmount() {
  console.log('[vue] unmount')
  app?.unmount()
  app = null
  router = null
}

// 可选: update 钩子
export async function update(props: any) {
  console.log('[vue] update', props)
}

```

```
// vite.config.ts
import qiankun from 'vite-plugin-qiankun'

```

```
export default defineConfig({
  plugins: [
    vue(),
    qiankun('vue-app', {
      useDevMode: true,
    }),
  ],
  server: {
    port: 8081,
    cors: true,
    origin: 'http://localhost:8081',
  },
})

```

qiankun 特点

特性	说明
HTML Entry	直接加载子应用 HTML，解析其中的 JS/CSS
JS 沙箱	支持多种沙箱模式（Proxy / Snapshot）
样式隔离	Shadow DOM / Scoped CSS
预加载	支持子应用资源预加载
生命周期	bootstrap / mount / unmount / update

---

2. micro-app（京东）

基本信息

- 基于 Web Components 思想
- 使用自定义元素 `<micro-app>`
- 接入成本更低

主应用配置

```
// main.ts
import microApp from '@micro-zoe/micro-app'
```

```

microApp.start({
  // 全局配置
  lifeCycles: {
    created() {
      console.log('created')
    },
    beforemount() {
      console.log('beforemount')
    },
    mounted() {
      console.log('mounted')
    },
    unmount() {
      console.log('unmount')
    },
    error() {
      console.log('error')
    },
  },
  // 插件系统
  plugins: {
    modules: {
      'appname-child': [{
        loader(code) {
          // 可以修改子应用代码
          return code
        },
      }],
    },
  },
})

```

```
<!-- 主应用使用 -->
<template>
  <div>
    <h1>主应用</h1>
    <micro-app
      name="child-app"
```

```
      url="http://localhost:8081/"
      :data="{ msg: 'hello' }"
      @created="handleCreated"
      @mounted="handleMounted"
      @unmount="handleUnmount"
      @error="handleError"
    ></micro-app>
  </div>
</template>

```

子应用配置（几乎无改造）

```
// vite.config.ts
export default defineConfig({
  base: 'http://localhost:8081/',  // 设置为绝对路径
  server: {
    port: 8081,
    cors: true,
    headers: {
      'Access-Control-Allow-Origin': '*',
    },
  },
})

```

```
// main.ts - 监听卸载事件
window.addEventListener('unmount', () => {
  app.unmount()
})

```

micro-app 特点

特性	说明
零改造成本	子应用几乎不需要修改
组件化	像使用组件一样使用微前端
虚拟路由	自动处理路由冲突
数据通信	支持 data 属性传值
样式隔离	默认开启样式隔离

---

3. wujie（腾讯）

基本信息

- 基于 **iframe + Web Components**
- 利用 iframe 的天然隔离，解决 iframe 的体验问题
- 子应用运行在 iframe 中，但 DOM 渲染在主应用

原理

```
┌──────────────────────────────────────────┐
│                  主应用                    │
│  ┌──────────────────────────────────┐    │
│  │     Web Component 容器           │    │
│  │  ┌────────────────────────┐     │    │
│  │  │  子应用的 DOM（可见）      │     │    │
│  │  └────────────────────────┘     │    │
│  └──────────────────────────────────┘    │
│                                          │
│  ┌──────────────────────────────────┐    │
│  │  隐藏的 iframe（JS 运行环境）     │    │
│  │  - window / document 代理        │    │
│  │  - JS 在此执行，DOM 操作代理出去  │    │
│  └──────────────────────────────────┘    │
└──────────────────────────────────────────┘

```

主应用配置

```
// main.ts
import { setupApp, bus } from 'wujie'

// 预加载
setupApp({
  name: 'vue-child',
  url: 'http://localhost:8081/',
  exec: true, // 预执行
})

```

```
<template>
  <WujieVue
    name="vue-child"
    url="http://localhost:8081/"
    :props="{ token: 'xxx' }"
    @created="handleCreated"
  />
</template>

<script setup>
import WujieVue from 'wujie-vue3'
</script>

```

wujie 特点

特性	说明
iframe 隔离	天然的 JS/CSS 隔离
DOM 同层渲染	解决 iframe 的弹窗、滚动问题
保活模式	切换时保留子应用状态
通信	基于 props、eventBus
预加载/预执行	性能优化

---

4. 框架对比

特性	qiankun	micro-app	wujie
实现原理	single-spa + 沙箱	Web Components	iframe + Web Components
JS 隔离	Proxy 沙箱	Proxy 沙箱	iframe 天然隔离
CSS 隔离	Shadow DOM / Scoped	默认 Scoped	iframe 天然隔离
接入成本	中等	低	低
子应用改造	需要导出生命周期	几乎无需改造	几乎无需改造
兼容性	IE11+（降级）	不支持 IE	良好
维护团队	蚂蚁	京东	腾讯
成熟度	最成熟	成熟	较新

五、核心技术原理

1. JS 沙箱隔离

问题

子应用可能修改全局变量（`window.xxx = ...`），导致污染主应用或其他子应用。

方案一：快照沙箱（SnapshotSandbox）

```
// 原理：切换前保存 window 快照，切换后恢复
class SnapshotSandbox {
  windowSnapshot = {}
  modifyPropsMap = {}

  // 激活沙箱
  active() {
    // 1. 保存当前 window 快照
    for (const prop in window) {
      this.windowSnapshot[prop] = window[prop]
    }

    // 2. 恢复上次的修改
```

```
    Object.keys(this.modifyPropsMap).forEach((prop) => {
      window[prop] = this.modifyPropsMap[prop]
    })
  }

  // 失活沙箱
  inactive() {
    for (const prop in window) {
      if (window[prop] !== this.windowSnapshot[prop]) {
        // 记录修改
        this.modifyPropsMap[prop] = window[prop]
        // 恢复原值
        window[prop] = this.windowSnapshot[prop]
      }
    }
  }
}

```

缺点：不支持多个子应用同时运行。

方案二：代理沙箱（ProxySandbox）

```
// 原理：使用 Proxy 代理 window，每个子应用有独立的 fakeWindow
class ProxySandbox {
  proxyWindow: Window
  isRunning = false

  constructor() {
    const fakeWindow = Object.create(null)

    this.proxyWindow = new Proxy(fakeWindow, {
      get(target, prop) {
        // 优先从 fakeWindow 取，取不到再从真实 window 取
        if (prop in target) {
          return target[prop]
        }
        const value = window[prop]
        // 函数需要绑定到真实 window
        if (typeof value === 'function' && !value.prototype) {
```

```
          return value.bind(window)
        }
        return value
      },

      set(target, prop, value) {
        // 只修改 fakeWindow，不影响真实 window
        target[prop] = value
        return true
      },

      has(target, prop) {
        return prop in target || prop in window
      },
    })
  }

  active() {
    this.isRunning = true
  }

  inactive() {
    this.isRunning = false
  }
}

```

优点：支持多个子应用同时运行。

方案三：iframe 沙箱

```
// 原理：子应用的 JS 在 iframe 中执行
const iframe = document.createElement('iframe')
iframe.src = 'about:blank'
document.body.appendChild(iframe)

const sandboxWindow = iframe.contentWindow
// 子应用的代码在 sandboxWindow 中执行

```

优点：最彻底的隔离。

缺点：iframe 的各种问题。

---

2. CSS 样式隔离

问题

子应用的 CSS 可能影响主应用或其他子应用。

方案一：Shadow DOM

```
// 创建 Shadow DOM，样式完全隔离
const container = document.getElementById('subapp')
const shadow = container.attachShadow({ mode: 'open' })
shadow.innerHTML = `
  <style>
    /* 这里的样式不会影响外部 */
    .btn { color: red; }
  </style>
  <div class="app">子应用内容</div>
`

```

缺点：

- 弹窗（挂载到 body）无法继承样式
- 部分组件库不兼容

方案二：Scoped CSS（添加前缀）

```
// 给子应用的所有 CSS 选择器添加前缀
// 原始 CSS
// .btn { color: red; }

// 转换后
// div[data-qiankun="subapp"] .btn { color: red; }

function scopedCSS(styleText, appName) {
  const prefix = `div[data-qiankun="${appName}"]`
  return styleText.replace(
```

```
    /([^{}]+)\{/g,
    (match, selector) => {
      const scoped = selector
        .split(',')
        .map((s) => `${prefix} ${s.trim()}`)
        .join(',')
      return `${scoped}{`
    }
  )
}

```

方案三：CSS Modules / CSS-in-JS

```
/* CSS Modules 编译后会生成唯一类名 */
.btn { } /* → .btn_x7d3f */

```

方案四：命名空间约定

```
/* 人工约定，所有子应用样式加前缀 */
.subapp-a .btn { }
.subapp-b .btn { }

```

---

3. 应用间通信

方案一：Props 传递

```
// 主应用
registerMicroApps([
  {
    name: 'sub-app',
```

```
    entry: '//localhost:8081',
    container: '#subapp',
    activeRule: '/sub',
    props: {
      token: 'xxx',
      userInfo: { name: 'admin' },
      onLogout: () => { /* ... */ },
    },
  },
])

// 子应用
export async function mount(props) {
  console.log(props.token)
  console.log(props.userInfo)
  props.onLogout()
}

```

方案二：全局状态管理

```
// qiankun 提供的全局状态
import { initGlobalState } from 'qiankun'

// 主应用
const actions = initGlobalState({
  user: null,
  token: '',
})

actions.onGlobalStateChange((state, prev) => {
  console.log('状态变化', state, prev)
})

actions.setGlobalState({ token: 'new-token' })

// 子应用
export async function mount(props) {
  props.onGlobalStateChange((state) => {
```

```
    console.log('子应用收到状态', state)
  })

  props.setGlobalState({ user: { name: 'admin' } })
}

```

方案三：自定义事件

```
// 发送事件
window.dispatchEvent(new CustomEvent('micro-event', {
  detail: { type: 'LOGIN', data: { token: 'xxx' } },
}))

// 监听事件
window.addEventListener('micro-event', (e) => {
  console.log(e.detail)
})

```

方案四：发布订阅

```
// 简单的 EventBus
class EventBus {
  events = new Map()

  on(event, callback) {
    if (!this.events.has(event)) {
      this.events.set(event, [])
    }
    this.events.get(event).push(callback)
  }

  emit(event, data) {
    const callbacks = this.events.get(event)
    callbacks?.forEach((cb) => cb(data))
  }
```

```

  off(event, callback) {
    const callbacks = this.events.get(event)
    if (callbacks) {
      const index = callbacks.indexOf(callback)
      if (index > -1) callbacks.splice(index, 1)
    }
  }
}

// 挂载到 window，所有应用共享
window.__MICRO_EVENT_BUS__ = new EventBus()

```

---

4. 路由处理

路由劫持原理

```
// 监听 history 变化
const originalPushState = history.pushState
const originalReplaceState = history.replaceState

history.pushState = function(...args) {
  originalPushState.apply(this, args)
  // 触发自定义事件，通知微前端框架
  window.dispatchEvent(new PopStateEvent('popstate'))
}

history.replaceState = function(...args) {
  originalReplaceState.apply(this, args)
  window.dispatchEvent(new PopStateEvent('popstate'))
}

// 监听路由变化
window.addEventListener('popstate', () => {
  // 根据当前 URL 匹配并加载对应子应用
  const path = window.location.pathname
  // matchApp(path)
```

```
})

```

子应用路由配置

```
// Vue Router
const router = createRouter({
  history: createWebHistory(
    // 子应用需要加上自己的 base 路径
    window.__POWERED_BY_QIANKUN__ ? '/sub-app/' : '/'
  ),
  routes,
})

```

---

5. 应用生命周期

```
                        ┌──────────────┐
                        │  加载子应用  │
                        └──────┬───────┘
                               │
                               ▼
                        ┌──────────────┐
          ┌─────────────│  bootstrap   │ (初始化，只执行一次)
          │             └──────┬───────┘
          │                    │
          │                    ▼
          │             ┌──────────────┐
          │   ┌─────────│    mount     │ (挂载，每次激活执行)
          │   │         └──────┬───────┘
          │   │                │
          │   │                ▼
          │   │         ┌──────────────┐
          │   │         │   运行中     │
          │   │         └──────┬───────┘
          │   │                │
```

```
        |  |                ▼
        |  |         ┌──────────────┐
        |  └────────►|   unmount    | (卸载，每次失活执行)
        |            └──────┬───────┘
        |                   |
        |                   | (再次激活)
        |                   ▼
        └──────────────────┘

```

六、常见问题与解决方案

问题	原因	解决方案
子应用静态资源 404	publicPath 配置错误	配置为绝对路径或 `window.__INJECTED_PUBLIC_PATH_BY_QIANKUN__`
子应用路由跳转失效	路由 base 未配置	子应用路由加上 activeRule 作为 base
样式冲突	CSS 未隔离	开启 Shadow DOM 或 Scoped CSS
全局变量污染	沙箱未生效	检查沙箱配置
子应用白屏	生命周期未正确导出	检查 bootstrap/mount/unmount 导出
跨域问题	子应用未配置 CORS	子应用服务器配置 CORS 响应头
弹窗定位错误	Shadow DOM 导致	使用 experimentalStyleIsolation 或挂载到 body

七、面试高频考点

考点	关键答案
什么是微前端?	将前端拆分成独立应用，运行时组合
为什么需要微前端?	解决巨石应用问题：独立开发部署、技术栈无关、增量升级
微前端实现方案?	iframe、npm、路由分发、Web Components、JS 沙箱+动态加载、Module Federation
qiankun 原理?	基于 single-spa，HTML Entry 加载，Proxy 沙箱隔离
JS 沙箱怎么实现?	快照沙箱（diff window）、代理沙箱（Proxy fakeWindow）、iframe 沙箱
CSS 隔离怎么实现?	Shadow DOM、Scoped CSS（选择器加前缀）、CSS Modules
应用间怎么通信?	props、全局状态、自定义事件、EventBus
生命周期有哪些?	bootstrap（初始化）、mount（挂载）、unmount（卸载）
qiankun vs micro-app?	qiankun 更成熟需改造；micro-app 接入成本低
Module Federation 是什么?	Webpack 5 模块联邦，运行时共享模块